

Phase 2: PID Attitude Controller

Phase 2 takes the validated quadcopter model from Phase 1 and wraps a controller around it for the first time. Until this point, the drone in simulation had no way of correcting itself — a torque applied to it kept rotating the drone with nothing pulling it back. Phase 2 implements a **PID controller**, the same type used inside almost every commercial flight controller today, applied to hold the drone at a desired roll, pitch, and yaw angle.

There are two reasons this phase exists. First, **practical**: a working PID controller gives the project a functioning closed-loop system that can be simulated, tuned, and later carried over to real hardware. Second, **comparative**: PID is the baseline that the Sliding Mode Controller in Phase 3 will be measured against. Every number produced here — overshoot, settling time, and disturbance recovery time — becomes a target the SMC is expected to beat.



Recap: What Phase 1 Provides

Phase 2 depends completely on two files from Phase 1: `parameters.m` and `dynamics.m`.

`parameters.m` defines the physical properties of the drone — mass, arm length, the three moments of inertia, and aerodynamic coefficients — packed into a single struct `p`. `dynamics.m` takes the current state and commanded torques, returning how fast every state is changing using the nonlinear Newton-Euler rotational equations.

The state vector inherited from Phase 1 has six elements, listed below for reference since every file in Phase 2 builds directly on this same indexing.

Phase 2 does **not** change `dynamics.m` in any way. Every equation inside it, including the gyroscopic coupling terms, stays exactly as derived and validated in Phase 1. What Phase 2 adds is a way to compute the torque inputs `tau_phi`, `tau_theta`, and `tau_psi` automatically, based on the gap between where the drone currently is and where it needs to be.

The Three Terms of a PID Controller

A PID controller builds its output torque from three separate terms added together, each reacting to a different aspect of the tracking error (desired angle minus actual angle).

Proportional Term

Reacts to how large the current error is. If the drone is tilted ten degrees away from where it should be, this term produces a torque proportional to that ten degrees — like a stretched spring pulling back harder the further it is stretched.

Integral Term

Reacts to how long and how consistently an error has persisted. If a constant force keeps the drone slightly tilted, the proportional term alone will settle at a fixed output leaving a small permanent tilt. The integral term keeps accumulating until that residual tilt is driven to zero.

Derivative Term

Reacts to how fast the error is currently changing — in this project, the drone's angular rate directly. It behaves like a damper, resisting fast motion and stopping the proportional term from pushing the drone past its target and oscillating.

Written as a single equation, the torque commanded on one axis is:

Deriving the Gains from the Physical Model

Rather than guessing values for K_p , K_i , and K_d by trial and error, the gains were derived directly from the Phase 1 physical model. At hover, all three angles and angular rates are zero. The coupling term disappears and the roll equation collapses to a simple **double integrator**.

A double integrator plant controlled with proportional and derivative feedback has a well-known closed-loop characteristic equation. This can be matched directly against the standard second-order form $s^2 + 2\zeta\omega_n s + \omega_n^2$, where ω_n is the natural frequency and ζ is the damping ratio. Matching coefficients gives two direct formulas for the gains.

Roll & Pitch

$$\omega_n = 5 \text{ rad/s}$$

$$\zeta = 0.8$$

Yaw

$$\omega_n = 3 \text{ rad/s (lower, since } I_{zz} \text{ is larger)}$$

$\zeta = 0.8$ — close to critically damped, giving fast response with under 2% overshoot in theory.

Integral Term, Augmented State & Anti-Windup

1 Why the Integral Term Matters

The plant itself, being a double integrator (Type 2 system), already gives zero steady-state error for a step reference with pure PD control. The real purpose of K_i is

rejecting constant external disturbances.

If a constant torque acts on the drone from wind or a shifted payload, pure PD control settles at a small nonzero tilt permanently. The integral term removes this residual tilt over time — exactly what Test 2 demonstrates.

2 The Augmented State Vector

The integral of the error is treated as a genuine state with its own differential equation. This extends the original six-element state vector into a **nine-element augmented state vector**, with three extra states holding the running integral for roll, pitch, and yaw. `ode45` then integrates these with the same numerical accuracy as the six attitude states.

3 Preventing Integral Windup

If the integral keeps growing while the output is clipped at its maximum, it builds up a hidden value that later causes noticeable overshoot — called **integral windup**. The fix is **conditional integration**: before the integral state grows, a check is made — is the unsaturated torque already beyond the limit, and is the error still pushing further in that direction? If both are true, the integral's derivative is frozen at zero. With the gains chosen, torques stay well under the 0.5 N·m limit, so this protection does not trigger for current test cases.

Code Walkthrough: parameters.m & pid_gains.m

Phase 2 consists of four files in addition to the two carried over from Phase 1. Each has a single, clearly defined job.

parameters.m — the g0 Fix

Unchanged in its physics from Phase 1, but one important fix was made. The original file used a bare variable `g` for gravity (9.81 m/s^2). Phase 2 introduces its own struct also called `g` for PID gains (`g.roll`, `g.pitch`, `g.yaw`). When both existed under the same name, MATLAB produced the error *dot indexing is not supported for variables of this type*. The fix: rename the internal gravity variable to `g0` inside `parameters.m`. The struct field `p.g` was left unchanged, so nothing outside needed to be touched.

pid_gains.m

Computes the nine gain values using the pole placement formulas from Section 3.2 and packs them into struct `g`. Must be run after `parameters.m`, since it reads `p.lxx`, `p.lyy`, and `p.lzz` directly. The same block is repeated for pitch (using `p.lyy`) and yaw (using `p.lzz` and a lower natural frequency of 3 rad/s). Finishes by printing a formatted summary of all nine values to the console.

Code Walkthrough: pid_closed_loop.m

This is the central file of Phase 2 — the ODE function passed to `ode45`, combining the PID controller with the Phase 1 plant into a single closed-loop system. It works through nine clearly separated steps:

01

Unpack state, reference, and compute tracking errors

02

Compute raw, unsaturated PID output for each axis

Derivative term taken directly from measured angular rate — avoids a large torque spike the instant a new reference is commanded.

03

Clip raw torque to actuator limit

04

Add external disturbance torque (once start time reached)

05

Pass torques into `dynamics.m` from Phase 1, unchanged

06

Implement anti-windup logic — freeze integral if saturated

07

Assemble all nine derivatives into a single column vector for `ode45`

Code Walkthrough: `step_metrics.m` & `simulate_pid.m`

`step_metrics.m`

A small, reusable helper that measures three standard control engineering quantities from any time response: **rise time**, **settling time**, and **overshoot**. Does not depend on the Control System Toolbox — runs on any MATLAB installation.

- Rise time: normalise response, locate first crossings at 10% and 90% of target
- Settling time: last moment response leaves a band around final value
- Overshoot: highest point beyond final value, as percentage of step size

Used again unchanged in Phase 3, making a fair, consistent comparison between PID and SMC possible.

`simulate_pid.m`

The script actually run to produce results. Loads parameters and gains, executes three test cases, prints a full validation report, and generates three figures.

- **Test 1:** Clean 10° roll step, no disturbance, no initial rate — isolates roll axis, produces baseline PID numbers
- **Test 2:** Hold level, inject 0.01 N·m disturbance on roll axis at $t=2s$
- **Test 3:** Roll and pitch to 15° simultaneously, with initial yaw rate of 0.5 rad/s — activates gyroscopic coupling terms

Closes by comparing coupled Test 3 results against clean Test 1 baseline, printing whether performance degraded under coupling — concrete evidence motivating the move to SMC in Phase 3.

Validation Results

Results expected from running `simulate_pid.m`, derived analytically from the gain values and Phase 1 physical parameters. Test 1 is close to an exact linear system near hover, so its predicted numbers are reliable. Tests 2 and 3 involve nonlinear behaviour and are reported as approximate, expected ranges.

Test 1: Step Response

Follows directly from damping ratio 0.8, giving roughly **1.5% overshoot** and settling time of about **1 second**. Steady-state error stays negligible because the plant is already a Type 2 system — reaches target with zero error even before the integral term acts.

Test 2: Disturbance Rejection

With 0.01 N·m disturbance and proportional gain 0.1214, a pure P controller would settle at $\sim 4.7^\circ$ static tilt. With derivative damping included, maximum deviation expected in the range of **4–5°** shortly after disturbance begins, before the integral term drives tilt back toward zero over roughly **1–2 seconds**. This recovery time is the specific number Phase 3 is expected to improve on.

Test 3: Coupled Axes

With roll and pitch at 15° and initial yaw rate of 0.5 rad/s, the roll and pitch equations pick up a nonzero coupling term. Expected to show as a **small increase in overshoot** on both axes compared with Test 1, along with a brief yaw drift of a few degrees. The yaw axis brings itself back toward zero on its own via its own PD terms.

Once `simulate_pid.m` has actually been run, the approximate figures should be replaced with exact numbers printed to the console, which include a direct pass/fail check against the official target specification for every test.

Conclusion and Transition to Phase 3

Phase 2 delivers a complete, working PID attitude controller built directly on top of the Phase 1 model, with gains derived analytically rather than guessed, an augmented state vector that lets the integral term be integrated with the same accuracy as the rest of the system, and anti-windup protection built in from the start.

More importantly, Phase 2 produces the specific numbers that give the rest of the project its purpose. The step response in Test 1 shows that PID works well under ideal, decoupled conditions. The disturbance recovery time from Test 2 and the coupling degradation shown in Test 3 both point at the same underlying limitation: **a controller designed around a linearised model at hover has no way of accounting for the nonlinear terms that appear the moment the drone moves away from that single operating point.**

- This is exactly the gap that Sliding Mode Control is designed to close. Phase 3 uses the same Phase 1 plant and the same `step_metrics.m` evaluation code, which means every number produced in this report becomes a direct, fair comparison point once the SMC controller is built and tested under the same three conditions.